

CareerHub — Wiring the UI to the Real Backend

Connecting the Next.js frontend job board to the live ASP.NET + Postgres API · Generated 23 June 2026

STATUS: WIRED UP & VERIFIED. The homepage at `http://localhost:3000` now loads **real jobs from Postgres** via the backend at `http://localhost:8080`. The sections below record what changed and the manual steps you need to know to run, prove, and maintain it.

1 · What was changed (in this repo only)

File	Change
<code>.env.local</code>	Added <code>NEXT_PUBLIC_API_BASE_URL=http://localhost:8080</code> — the live backend. Left <code>NEXT_PUBLIC_API_URL</code> unset on purpose (keeps applications / recruiter / auth on the in-app routes).
<code>src/lib/api.ts</code>	Added a single <code>JOBS_API_BASE</code> constant that reads the new env var. <code>fetchJobs</code> and <code>fetchJobById</code> now call the real backend; every other function is untouched. No hardcoded URLs.

Not changed (and why): the TypeScript types in `src/types/index.ts` already match the backend's DTO *exactly* (same fields, same camelCase, same nullable salaries), so no type edits were needed. `page.tsx`, `providers.tsx`, the skeleton loader, React Query Devtools and the error UI were all left intact.

2 · Contract check — backend is the source of truth

Pulled live from `http://localhost:8080/openapi/v1.json` and a real `GET /api/jobs` response:

Aspect	Backend (live)	Your TS type	Verdict
List item fields	id, title, location, type, salaryMin, salaryMax, status, createdAt, expiresAt, companyId, companyName, companyCity, minimumExperienceYears, responsibilities, skills, applicantCount	<code>JobListingResponse</code>	Identical
Casing	camelCase	camelCase	Match — no transform needed
Envelope	data, page, pageSize, totalCount, totalPages, hasNextPage, hasPreviousPage, links	<code>PagedResponse<T></code>	Identical
Detail extras	description, minimumRequirements, companyProvince, companyWebsite	<code>JobListingDetailResponse</code>	Identical

3 · Why direct calls (not a proxy)

The browser calls the backend straight from TanStack Query's `queryFn`, because the backend already returns CORS headers for the frontend origin (verified: an `OPTIONS` preflight returns `204` with `Access-Control-Allow-Origin: http://localhost:3000`). The env var is `NEXT_PUBLIC_`-prefixed, i.e. intended for the browser. No proxy route was needed, so nothing forwards or hides the URL — simplest correct choice for local dev.

4 · Manual steps YOU need to do / know

A. Prove the data is real (the acceptance test)

1. Open the API docs UI: `http://localhost:8080/scalar/v1`
2. Find `POST /api/Jobs`, fill in a new job (title, company, etc.) and send it.
3. Refresh `http://localhost:3000` — your new job appears in the grid. That round trip through Postgres is the proof it is not mock data.

B. Starting everything from cold (after a reboot)

```
# 1. Backend (API + Postgres) – from the backend repo folder:
cd C:\Users\Xolisa\assignment-3.2
docker compose up -d

# 2. Frontend – from this repo:
cd C:\Users\Xolisa\careerhub-1.3
npm run dev
```

The backend image takes a few minutes to build the first time (.NET 10 publish). After that it starts in seconds.

C. If the board shows the error panel / 500s: the Postgres container can drop into "recovery mode" when the machine is under heavy load (it happened once during the .NET build). The API's connection pool then holds dead connections. Fix:

```
# wait for Postgres to report healthy, then bounce the API:
docker restart careerhub24-api
```

Give it ~30–60 s to re-run migrations and start listening, then refresh the page.

D. Known limitation — pagination

The backend holds ~**600 jobs** and paginates 20 at a time. The board currently requests `?page=1&pageSize=50`, so it shows the first 50 and the search/sort/filter operate on those 50. Wiring real page navigation (using the `links.next` the API already returns) is a clean follow-up but was out of scope for "show live jobs".

E. Application & recruiter flows still use the in-app store

Submitting an application and posting jobs from the recruiter screens still write to this Next.js app's own `/api` routes, *not* the real backend — the live API exposes those under a different shape (e.g. `POST /api/v1/jobs/{id}/applications`). This is deliberate: only the job board was in scope. Wiring those to the real backend is a separate task.

5 · Verification performed

Check	Result
Backend URL the browser hits	<code>GET http://localhost:8080/api/jobs?page=1&pageSize=50</code>
Backend response	<code>HTTP 200 · 50 items · totalCount 600 · real SA employers (Standard Bank, Discovery, Capitec)</code>
CORS for localhost:3000	<code>Allowed (preflight 204 + Access-Control-Allow-Origin)</code>
Frontend homepage	<code>HTTP 200, renders and hydrates the grid from the API</code>
TanStack Query / skeleton / devtools / error UI	<code>Preserved (queryKey <code>["jobs"]</code> , default staleTime)</code>

CareerHub · Backend: `C:\Users\Xolisa\assignment-3.2` (containers `careerhub24-api` :8080, `careerhub24-pg` :5544). Frontend: `C:\Users\Xolisa\careerhub-1.3` (`npm run dev` :3000).